



兰州工业学院

实 验 报 告

(2023 —2024 学年第 1 学期)

课程名称 面向对象程序设计

专业班级 专升本电信 23

学 号 ZB2305010116

姓 名 姜 勇

实验项目	实验二 类的继承		
实验时间	2023 年 10 月 23 日	实验地点	信息与通信工程专业实验室 (DSP)
一、实验目的 1. 学习定义和使用类的继承关系，定义派生类； 2. 熟悉不同继承方式下对基类成员的访问控制； 3. 掌握在派生类中初始化基类成员的方法； 4. 掌握 C++ 函数的重载机制； 5. 掌握使用虚函数实现动态多样性。 二、实验环境 硬件：计算机一台； 软件：VC++ 6.0 或 Code::Blocks。 三、实验内容 1. 定义一个人员类 Person，包括姓名、性别、年龄等数据成员和用于输入、输出的成员函数，设计构造函数完成数据初始化操作。在此基础上派生出学生类 Student 类（增加成绩）和教师类 Teacher（增加教龄），并实现对学生和教师信息的输入和输出。 人员类 <pre> #ifndef PERSON_H #define PERSON_H #include "string" using namespace std; //人员类 class Person { public: Person(); //默认构造函数 Person(string n, string s, int a, string p); //有参构造函数 </pre>			

```

        void input_info(); //定义输入输出
        void output_info();
protected:
        string name; //姓名
        string sex; //性别
        int age; //年龄
        string phone; //电话

};

//学生类
class Student : public Person //继承了 Person 类的所有成员变量和成员函数。
{
private:
        float score; //成绩
public:
        Student(); //默认构造函数
        Student(string n, string s, int a, string p, float sc); //有参构造函数
        void input_info(); //定义输入输出
        void output_info();

};

//教师类
class Teacher : public Person{
private:
        int addyear; //教龄
public:
        Teacher(); //默认构造函数

```

```

    Teacher(string n,string s,int a,string p,int y);//有参构造函数

    void input_info();//定义输入输出

    void output_info();
};

#endif // PERSON_H

成员函数的实现

#include "Person.h"
#include "iostream"
using namespace std;
//人员类成员函数实现
Person::Person() {} //默认构造函数
Person::Person(string n,string s,int a,string p)//有参构造函数使用参
数的值初始化
{
    name=n;
    sex=s;
    age=a;
    phone=p;
}

void Person::input_info()
{
    cout << "请输入姓名：";
    cin>>name;
    cout << "请输入性别：";
    cin>>sex;
    if (sex == "男" || sex == "女") {}
    else {

```

```

        cout << "性别错误，请重新输入。" << endl;

        cout << "请输入性别：";

        cin>>sex;

    }

    cout << "请输入年龄：";

    cin>>age;

    if (age >= 18 && age <= 26) {

    } else {

        cout << "年龄超出有效范围，请重新输入。" << endl;

        cout << "请输入年龄：";

        cin>>age;

    }

    cout << "请输入电话：";

    cin>>phone;

}

void Person::output_info()
{

    cout << "姓名：" << name << endl;

    cout << "性别：" << sex << endl;

    cout << "年龄：" << age << endl;

    cout << "电话：" << phone << endl;

}

//学生类的成员函数实现
Student::Student() {}

//在函数体内部，使用参数的值调用父类 Person 的有参构造函数来初始化继承的成员变量

//使用参数的值来初始化本类的成员变量 score

```

```

Student::Student(string n,string s,int a,string p,float
sc):Person(n,s,a,p)
{
    score=sc;
}
void Student:: input_info()
{
    Person::input_info();
    cout << "请输入成绩：";
    cin >> score;
    if (score >= 0 && score <= 100) {
    } else {
        cout << "成绩超出有效范围，请重新输入。" << endl;
        cout << "请输入成绩：";
        cin>>score;
    }
}
void Student::output_info()
{
    Person::output_info();
    cout << "成绩：" << score << endl;
}
Teacher::Teacher() {}
Teacher::Teacher(string n,string s,int a,string p,int
y):Person(n,s,a,p)
{
    addyear=y;
}

```

```

}

void Teacher::input_info()
{
    Person::input_info();
    cout << "请输入教龄：";
    cin >> addyear;
}

void Teacher::output_info()
{
    Person::output_info();
    cout << "教龄：" << addyear << "年" << endl;
}

```

主函数

```

#include <iostream>
#include "Person.h"
using namespace std;

int main()
{
    Student stu;//创建了一个 Student 对象 stu
    cout << "学生信息如下：\n";
    stu.input_info();
    cout << endl;
    stu.output_info();
    cout << endl;
    Teacher tch;//创建了一个 Teacher 对象 tch
    cout << "教师信息如下：\n";
    tch.input_info();
}

```

```

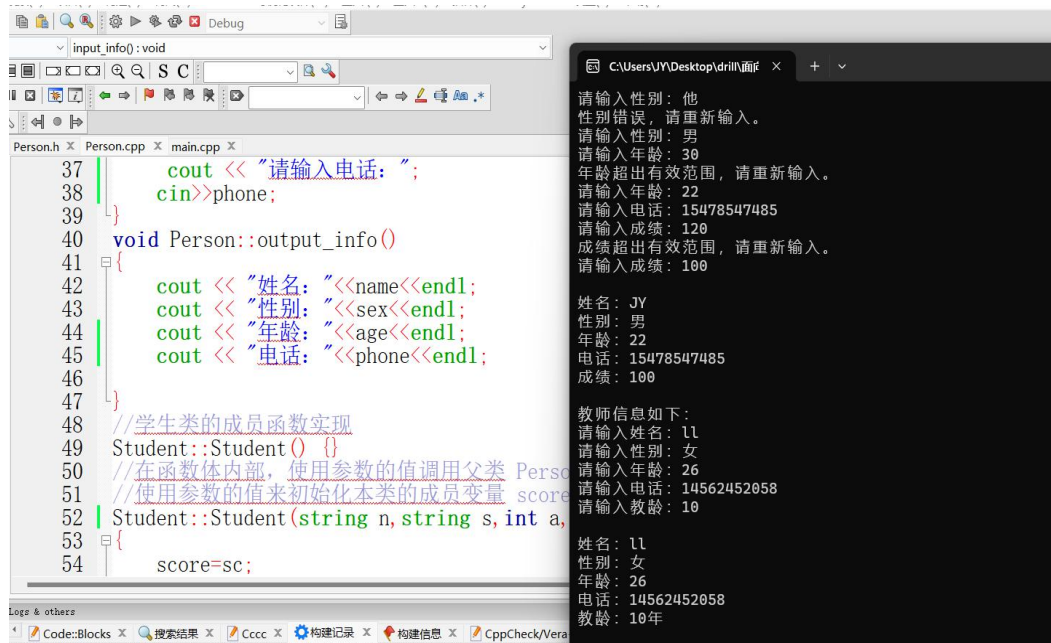
    cout << endl;

    tch.output_info();

    return 0;

}

```



2. 定义一个抽象类 Shape，包含两个纯虚函数 Area(计算面积)和 SetShape(重设形状大小)。然后派生出三角形类 Triangle、矩形 Rect 类、圆 Circle 类，分别求面积。

抽象类

```

#ifndef SHAPE_H
#define SHAPE_H
#include "string"
using namespace std;
class Shape
{
public:
    Shape(); //默认构造函数声明
    virtual float Area()=0; //面积

```



```

        virtual void setShape(float size)=0;//设置尺寸
    };

class Triangle : public Shape
{
private:
    float base;//底
    float height;//高
public:
    Triangle(float b, float h);
//实现了 Shape 类的成员函数
    float Area();
    void setShape(float size);
    string getshape();
};

class Rect : public Shape
{
private:
    float width;//宽
    float height;//高
public:
    Rect(float w, float h);
//实现了 Shape 类的成员函数
    float Area() ;
    string getshape();
    void setShape(float size);
};

class Circle : public Shape

```

```

{
private:
    float radius;//半径
public:
    Circle(float r);
//实现了 Shape 类的成员函数
    float Area();
    string getshape();
    void setShape(float size);
};
#endif // SHAPE_H
成员函数实现
#include "Shape.h"
#include "string"
#include "math.h"
using namespace std;
Shape::Shape() {}//Shape 类是一个抽象基类，它的构造函数为空。
//将底和高赋值给 base 和 height
Triangle::Triangle(float b, float h) : base(b), height(h) {}
//计算三角形面积
float Triangle::Area()
{
    return 0.5 * base * height;
}
//将底和高设置为相同的值
void Triangle::setShape(float size)
{

```

```

        base = size;
        height = size;
    }
    string Triangle::getshape()
    {
        return "这是三角形的";
    }
    //将宽和高赋值给 width 和 height
    Rect::Rect(float w, float h) : width(w), height(h) {}
    //计算矩形的面积
    float Rect::Area()
    {
        return width * height;
    }
    //将宽和高设置为相同的值
    void Rect::setShape(float size)
    {
        width = size;
        height = size;
    }
    string Rect::getshape()
    {
        return "这是矩形的";
    }
    //将半径赋值给 radius
    Circle::Circle(float r) : radius(r) {}
    //计算圆的面积

```

```

float Circle::Area()
{
    return 3.14*radius*radius ;
}

//半径设置为给定的值
void Circle::setShape(float size)
{
    radius = size;
}

string Circle::getshape()
{
    return "这是圆的";
}

```

主函数

```

#include <iostream>
using namespace std;
#include "Shape.h"
int main()
{
    Triangle triangle(3.0, 4.0);

    cout<<triangle.getshape()<<endl;
    cout << "三角形面积: " << triangle.Area() << endl;
    triangle.setShape(5.0);
    cout << "修改后的三角形面积: " << triangle.Area() << endl;
    cout<<endl;
    Rect rect(2.0, 3.0);
    cout<<rect.getshape()<<endl;
    cout << "矩形的面积: " << rect.Area() << endl;
    rect.setShape(4.0);
    cout << "修改后的矩形的面积: " << rect.Area() << endl;
    cout<<endl;
    Circle circle(2.0);
    cout<<circle.getshape()<<endl;
    cout << "圆的面积: " << circle.Area() << endl;
    circle.setShape(4.0);
}

```

```

    cout << "修改后的圆的面积：" << circle.Area() << endl;
    return 0;
}

```

The screenshot shows a C++ IDE with two windows. The left window, titled 'main.cpp', contains the following code:

```

cout<<triangle.getshape()<<endl;
cout << "三角形面积：" << triangle.getArea() << endl;
triangle.setShape(5.0);
cout << "修改后的三角形面积：" << triangle.getArea() << endl;
cout<<endl;
Rect rect(2.0, 3.0);
cout<<rect.getshape()<<endl;
cout << "矩形的面积：" << rect.getArea() << endl;
rect.setShape(4.0);
cout << "修改后的矩形的面积：" << rect.getArea() << endl;
cout<<endl;
Circle circle(2.0);
cout<<circle.getshape()<<endl;
cout << "圆的面积：" << circle.getArea() << endl;
circle.setShape(4.0);
cout << "修改后的圆的面积：" << circle.getArea() << endl;
return 0;
}

```

The right window shows the program's output:

```

这是三角形的
三角形面积： 6
修改后的三角形面积： 12.5

这是矩形的
矩形的面积： 6
修改后的矩形的面积： 16

这是圆的
圆的面积： 12.56
修改后的圆的面积： 50.24

Process returned 0 (0x0)   execution time : 0.019 s
Press any key to continue.

```

3. 分析以下程序代码，注释掉错误语句，并说明错误原因，分析程序的运行结果。

```

#include <iostream>

using namespace std;

class Base
{
private:
    int pvx;

protected :
    int pty;

public :
    float puf;

    void Setxy(int q, int u, float w)
    {

```

```

        pvx = q ; pty = u; puf = w;
    }

    void Output()
    {
        cout << "class Base output "<< endl;
        cout << "pvx= " << pvx << endl;
        cout << "pty= " << pty << endl;
        cout << "puf= " << puf << endl;
    }
};

class Derive : public Base
{
    private :
        float pvz;
    public :
        void Setvalue(int x, int y, float f, float z)
        {
            Setxy(x, y, f);
            pvx = x;
            pty = y;
            puf = f;
            pvz = z;
        }
        void Print()
        {
            cout << "pvx= " << pvx << endl;
            cout << "class Derive output " << endl;

```

```

        cout << "pty= "<<pty << endl;
        cout << "puf= "<<puf << endl;
        cout << "pvz= "<<pvz << endl;
    }
};

int main()
{
    Derive obj;
    obj.Setvalue(3,5, 8.9, 12.6);
    obj.Output();
    obj.Print();
    cout << "puf= " << obj.puf << endl;
    cout<<obj.pty<<endl;
    return 0;
}

```

分析得到程序的结果：

```

class Base output
pvx= 3
pty= 5
puf= 8.9
pvx= 3
class Derive output
pty= 5
puf= 8.9
pvz= 12.6
puf= 8.9
5

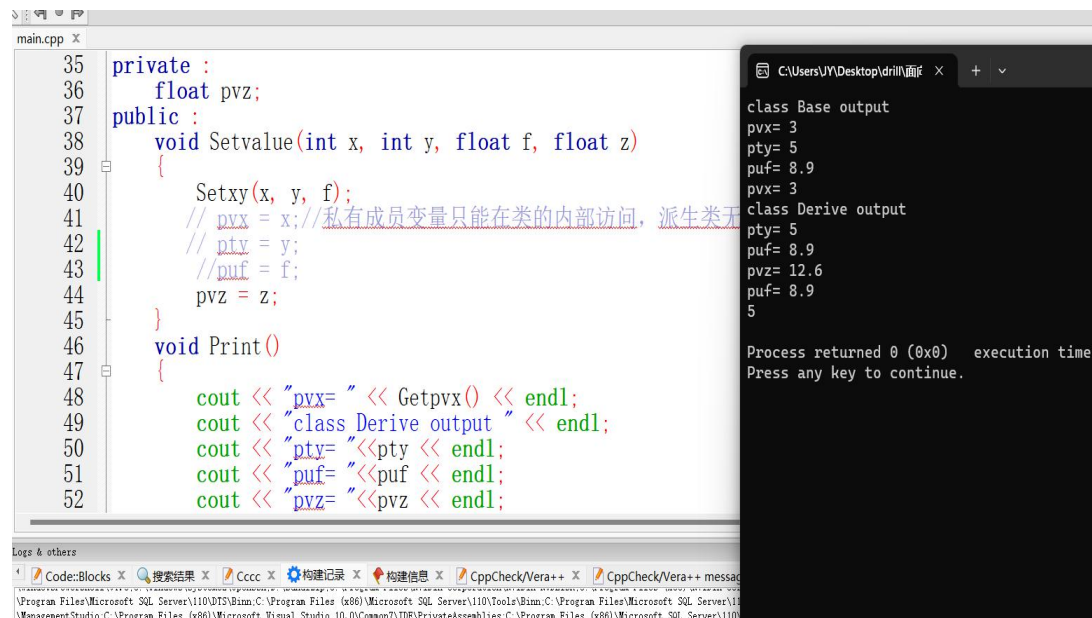
```

在 Derive 类的 Setvalue 方法中，不需要对 Base 类的成员变量 pvx、pty 和 puf 赋值，因为这些成员变量已经通过调用 Base 类的 Setxy 方法进行了赋值，可以将 setvalue 方法中的这 3 个赋值的成员变量删除

cout << "pvx= " << pvx<< endl; 因为 pvx 是 Base 类私有成员不能在类外直接访问，可以在 Base 类中添加一个访问 pvx 的公有成员函数来实现间接访问。也可以将 pvx 设置为公有成员

cout<<obj.pty<<endl; 对于受保护成员 pty，它可以被派生类访问，但不能直接通过对象进行访问。在 Derive 类中，可以通过继承自 Base 类的 pty 进行访问和操作。但在 main 函数中，由于 obj 是 Derive 类的对象，它无法直接访问或输出 Base 类的受保护成员 pty。创建一个 getpty 函数，用来间接访问 pty。

上机调试运行后得到结果：



The screenshot shows a C++ IDE with a file named 'main.cpp'. The code defines a 'Base' class with private members 'pvx', 'pty', and 'puf', and a 'Derive' class that inherits from 'Base'. The 'Derive' class has a 'Setvalue' method that calls 'Setxy' from the 'Base' class. The 'main' function creates a 'Derive' object and calls 'Print()' which outputs the values of 'pvx', 'pty', 'puf', and 'pvz'. The output window shows the results of the program execution.

```
private :
    float pvx;
public :
    void Setvalue(int x, int y, float f, float z)
    {
        Setxy(x, y, f);
        // pvx = x; // 私有成员变量只能在类的内部访问，派生类无法访问
        // pty = y;
        // puf = f;
        pvz = z;
    }
    void Print()
    {
        cout << "pvx= " << Getpvx() << endl;
        cout << "class Derive output " << endl;
        cout << "pty= " << pty << endl;
        cout << "puf= " << puf << endl;
        cout << "pvz= " << pvz << endl;
    }
}
```

```
class Base output
pvx= 3
pty= 5
puf= 8.9
pvx= 3
class Derive output
pty= 5
puf= 8.9
pvz= 12.6
puf= 8.9
5

Process returned 0 (0x0)   execution time
Press any key to continue.
```

四、实验总结

通过本次实验，我知道了如何定义基类和派生类，定义类成员可以定义为 public（公有）、protected（保护）、private（私有），在默认的情况下

派生类继承基类的访问权限，构造函数和析构函数，但不能继承基类的复制构造函数和移动构造函数。在派生类中，可以通过调用基类的构造函数来初始化基类的成员变量。C++支持多继承，即一个子类可以继承多个父类。多继承的语法与单一继承相同，子类的声明格式为：`class 子类名 : 访问修饰符 父类名 1, 访问修饰符 父类名 2, ...`。多继承的子类可以访问所有父类的公有和保护成员。

派生类可以重写（`override`）基类中的虚函数，以实现多态性。重写函数必须与基类中的虚函数具有相同的名称、参数列表和返回类型，并且必须使用 `override` 关键字标记。基类中的虚函数可以被派生类重写，通过基类指针或引用调用虚函数时，会根据对象的实际类型来确定调用哪个版本的虚函数。纯虚函数是在基类中声明但没有定义的虚函数，派生类必须实现纯虚函数。

实验二成绩评定表

序号	考核项目	分值分布	成绩
1	出勤与纪律情况	10	
2	实验任务完成情况	40	
3	实验报告质量	50	
总分			
指导教师签字			